

DAY 4 | PostgreSQL 18 backup and recovery

Complete dry run edition | 28 September 2026. Start at Lab 0. Run every command in your Ubuntu Bash terminal. Commands are plain selectable text. Paste one block at a time. Namespace postgres-ha; standalone pod backup-lab-0; Patroni cluster acid-postgres-ha.

Read this first: two separate databases

The pgBackRest repository /repo in backup-lab-0 contains backups of its own standalone PostgreSQL database, stanza lab. The three acid-postgres-ha pods are a different Patroni cluster. Backups from stanza lab cannot recover acid-postgres-ha. The standalone repository is on rancher.io/local-path storage on one Kubernetes node; it is a training backup, not protection against loss of that node.

Your GCP organization enforces constraints/iam.disableServiceAccountKeyCreation. Do not run gcloud iam service-accounts keys create, and do not configure pgBackRest with a JSON key that does not exist. A later GCS design needs a configured, verified workload identity for this non-GKE Kubernetes cluster. No GCS backup of acid-postgres-ha is claimed by this manual.

Lab 0 | Check your environment

```
NS=postgres-ha
CLUSTER=acid-postgres-ha
IMAGE=docker.io/eswaribala/ericpostgrerepo2026:pg18
echo "$NS $CLUSTER $IMAGE"
kubectl -n "$NS" get postgresql "$CLUSTER"
kubectl -n "$NS" get sts "$CLUSTER" \
  -o jsonpath='{.spec.updateStrategy.type}'{"\n"}
```

Variables such as NS exist only in your current Bash terminal. echo prints text; it does not edit a file or Kubernetes object. After opening another terminal, repeat the assignment block. OnDelete is expected for this StatefulSet.

```
kubectl -n postgres-ha get pods -o wide
kubectl -n postgres-ha get pvc
kubectl -n postgres-ha get configmap backup-lab-config
kubectl -n postgres-ha exec backup-lab-0 -- gosu postgres pgbackrest --stanza=lab info
```

Expected: backup-lab-0 and acid-postgres-ha-0/1/2 are Running. The lab stanza says status: ok and shows your existing full, differential and incremental backup labels. If a command fails, stop at that command; do not guess a new stanza or repository path.

Command anatomy

kubectl exec runs inside the named pod. The first -- ends kubectl options. gosu postgres runs the following program as the postgres Linux user, which pgBackRest requires. Do not add a second -- after postgres. pgbackrest --stanza=lab selects the standalone lab configuration.

Lab 1 | Logical backup of the live Patroni database

The leader can change. Ask PostgreSQL on each pod whether it is a replica. If local postgres authentication fails, stop and use a validated credential method rather than changing pg_hba to trust.

```
PRIMARY=
for pod in acid-postgres-ha-0 acid-postgres-ha-1 acid-postgres-ha-2; do
  role=$(kubectl -n postgres-ha exec "$pod" -c postgres -- \
    psql -U postgres -d postgres -Atqc 'SELECT pg_is_in_recovery()')
  echo "$pod in_recovery=$role"
  test "$role" = f && PRIMARY="$pod"
done
test -n "$PRIMARY" && echo "Primary: $PRIMARY"
```

Expected: only one pod prints f; its name is stored in PRIMARY. Re-run this after a failover. Output redirection > writes files on your Ubuntu machine, not in the pod or GCS.

```
mkdir -p backups/logical
kubectl -n postgres-ha exec "$PRIMARY" -c postgres -- \
  pg_dump -U postgres -Fc banking > backups/logical/banking.dump
kubectl -n postgres-ha exec "$PRIMARY" -c postgres -- \
  pg_dumpall -U postgres > backups/logical/all-databases.sql
test -s backups/logical/banking.dump
test -s backups/logical/all-databases.sql
ls -lh backups/logical
pg_restore -l backups/logical/banking.dump | head -25
```

banking.dump covers one database. all-databases.sql contains cluster roles and databases. A custom dump is not a WAL archive; it cannot support point in time recovery.

Optional logical restore check: use a disposable NEW database

```
kubectl -n postgres-ha exec "$PRIMARY" -c postgres -- \
  psql -U postgres -d postgres \
  -c 'CREATE DATABASE banking_restore_test;'
kubectl -n postgres-ha exec -i "$PRIMARY" -c postgres -- \
  pg_restore -U postgres -d banking_restore_test --no-owner \
  < backups/logical/banking.dump
kubectl -n postgres-ha exec "$PRIMARY" -c postgres -- \
  psql -U postgres -d banking_restore_test -c '\dt'
```

If that disposable database already exists, choose a new name consistently. Do not use banking as the restore target. Extensions or roles can require additional preparation.

Lab 2 | Independent physical base backup

This is separate from pgBackRest. A replication account streams a base backup from the Patroni primary Service to temporary space in a replica pod. Check space first: do not fill a running database pod filesystem.

```
kubect1 -n postgres-ha get secret | rg standby
kubect1 -n postgres-ha get svc acid-postgres-ha
REPL_SECRET=standby.acid-postgres-ha.credentials.postgresql.acid.zalan.do
kubect1 -n postgres-ha get secret "$REPL_SECRET"
REPL_USER=$(kubect1 -n postgres-ha get secret "$REPL_SECRET" \
-o jsonpath='{.data.username}' | base64 -d)
REPL_PASS=$(kubect1 -n postgres-ha get secret "$REPL_SECRET" \
-o jsonpath='{.data.password}' | base64 -d)
REPLICA=acid-postgres-ha-1
test "$REPLICA" != "$PRIMARY" || REPLICA=acid-postgres-ha-2
kubect1 -n postgres-ha exec "$REPLICA" -c postgres -- df -h /tmp
```

Gate: the standby Secret exists, REPL_USER is nonempty, and /tmp has sufficient free space for the entire cluster. If any condition fails, skip this optional exercise. Do not expose the password in screenshots.

```
kubect1 -n postgres-ha exec "$REPLICA" -c postgres -- \
env PGPASSWORD="$REPL_PASS" PGSSLMODE=require \
pg_basebackup -h acid-postgres-ha -p 5432 -U "$REPL_USER" \
-D /tmp/day4-basebackup -Ft -z -Xs -P
mkdir -p backups/physical
kubect1 -n postgres-ha cp -c postgres \
"$REPLICA:/tmp/day4-basebackup" \
backups/physical/day4-basebackup
unset REPL_PASS
find backups/physical/day4-basebackup -maxdepth 2 -type f
```

pg_basebackup refuses a nonempty target: do not rerun until you have selected another fresh path. The /tmp copy is temporary; the kubect1 cp result lives on Ubuntu. Keep a durable copy outside this local node if needed for disaster recovery.

Lab 3 | Check existing WAL archiving

```
kubectl -n postgres-ha exec "$PRIMARY" -c postgres -- \
  psql -U postgres -d postgres -c 'SHOW archive_mode;'
kubectl -n postgres-ha exec "$PRIMARY" -c postgres -- \
  psql -U postgres -d postgres -c 'SHOW archive_command;'
kubectl -n postgres-ha exec "$PRIMARY" -c postgres -- \
  psql -U postgres -d postgres -x \
  -c 'SELECT * FROM pg_stat_archiver;'
```

Record the current archive mode, command and archived WAL count. The operator image may already archive with WAL-G or another mechanism. Do not disable a working archive while evaluating a new repository.

Lab 4 | Verify the custom image and OnDelete behavior

```
kubectl -n postgres-ha get postgresql acid-postgres-ha \
  -o jsonpath='{.spec.dockerImage}{"\n"}'
kubectl -n postgres-ha get sts acid-postgres-ha \
  -o jsonpath='{.spec.template.spec.containers[0].image}{"\n"}'
for pod in acid-postgres-ha-0 acid-postgres-ha-1 acid-postgres-ha-2; do
  kubectl -n postgres-ha get pod "$pod" \
    -o jsonpath='{.metadata.name}{"\n"}'
  kubectl -n postgres-ha exec "$pod" -c postgres -- \
    pgbackrest version
done
```

Expected image: `docker.io/eswaribala/ericpostgrrepo2026:pg18`, spelled exactly. Confirm pod images and Ready in Lab 0. If all match, no edit is needed. To correct a mismatch, edit `spec.dockerImage` on the PostgreSQL custom resource and verify the generated StatefulSet; `metadata.name` stays `acid-postgres-ha`. Edit cancelled means nothing was saved. OnDelete does not support kubectl rollout status; replace outdated replicas individually only with a confirmed safe operator procedure.

Lab 5 | Back up existing standalone database

1. Review existing backups

```
kubectl -n postgres-ha exec backup-lab-0 -- gosu postgres pgbackrest --stanza=lab info
```

You already created a full backup on 26 September 2026 and successfully ran an incremental backup on 28 September 2026. An existing backup chain is sufficient for the next exercise. Repeating a full backup is optional for a fresh run.

2. Create a fresh full backup (optional)

```
kubectl -n postgres-ha exec backup-lab-0 -- \
  gosu postgres pgbackrest --stanza=lab --type=full backup
```

Expected: backup command end: completed successfully. Retention may expire older sets according to backup-lab-config; review info before choosing a restore set.

3. Make a differential backup

```
kubectl -n postgres-ha exec backup-lab-0 -- \
  gosu postgres pgbackrest --stanza=lab --type=diff backup
```

A differential backup contains changed files since the latest full backup.

4. Make an incremental backup

```
kubectl -n postgres-ha exec backup-lab-0 -- \
  gosu postgres pgbackrest --stanza=lab --type=incr backup
```

An incremental backup contains changed files since the previous successful backup. Your 28 September command already finished successfully with label 20260926-161449F_20260928-174132I.

5. Verify the backup chain and WAL archive

```
kubectl -n postgres-ha exec backup-lab-0 -- \
  gosu postgres pgbackrest --stanza=lab info
kubectl -n postgres-ha exec backup-lab-0 -- \
  gosu postgres pgbackrest --stanza=lab check
```

Expected: stanza status ok; backup list includes a full and newer differential and incremental labels; check completes successfully. Do not rely on the backup command alone as a restore test.

Lab 6 | Safe isolated physical restore

This exercise creates a NEW PVC called day4-lab-restore-data. The repository backup-lab-repo is mounted read-only in a second pod, on the same node. Never mount pgdata-acid-postgres-ha-0/1/2 or data-backup-lab-0 as the restore target. Run on your single-node local-path training cluster. If the pod cannot mount both volumes, stop; do not change node placement or PVC names to force it.

1. Verify configuration and discover image and node

```
kubectl -n postgres-ha get configmap backup-lab-config \
-o jsonpath='{.data.pgbackrest\.conf}'
kubectl -n postgres-ha get pod backup-lab-0 \
-o jsonpath='{.spec.containers[?(@.name=="postgres")].image}{"\n"}'
kubectl -n postgres-ha exec backup-lab-0 -- id postgres
```

Gate: ConfigMap output must contain [lab], repo1-path=/repo, and pg1-path. The image must be nonempty and contain pgBackRest and PostgreSQL 18. Note the uid and gid from id postgres. The next block reads them automatically. If the ConfigMap key is different, stop and inspect its keys rather than applying the manifest below.

2. Create a new empty recovery PVC

```
cat > day4-lab-restore-pvc.yaml <<'YAML'
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: day4-lab-restore-data
  namespace: postgres-ha
spec:
  accessModes: [ReadWriteOnce]
  resources:
    requests:
      storage: 3Gi
YAML
kubectl apply -f day4-lab-restore-pvc.yaml
kubectl -n postgres-ha get pvc day4-lab-restore-data
```

WaitForFirstConsumer storage may keep this PVC Pending until the toolbox pod is scheduled; this is normal. Make this PVC larger if your actual source database no longer fits in 3 GiB.

3. Prepare the isolated pod manifest

```
LAB_IMAGE=$(kubectl -n postgres-ha get pod backup-lab-0 \
-o jsonpath='{.spec.containers[?(@.name=="postgres")].image}')
LAB_NODE=$(kubectl -n postgres-ha get pod backup-lab-0 -o jsonpath='{.spec.nodeName}')
LAB_HOST=$(kubectl get node "$LAB_NODE" \
-o jsonpath='{.metadata.labels.kubernetes\.io/hostname}')
LAB_UID=$(kubectl -n postgres-ha exec backup-lab-0 -- id -u postgres)
LAB_GID=$(kubectl -n postgres-ha exec backup-lab-0 -- id -g postgres)
test -n "$LAB_IMAGE" && test -n "$LAB_HOST" && test -n "$LAB_UID" && test -n "$LAB_GID"
printf 'image=%s node=%s uid=%s gid=%s\n' \
"$LAB_IMAGE" "$LAB_HOST" "$LAB_UID" "$LAB_GID"
```

Lab 6 | Isolated restore (continued)

The next heredoc writes a YAML file in the current directory. The recovery container sleeps until you issue restore commands. Its image is the same image used by backup-lab-0.

```
cat > day4-lab-restore-pod.yaml <<YAML
apiVersion: v1
kind: Pod
metadata:
  name: day4-lab-restore
  namespace: postgres-ha
spec:
  restartPolicy: Never
  nodeSelector:
    kubernetes.io/hostname: "$LAB_HOST"
  initContainers:
    - name: prepare-directory
      image: "$LAB_IMAGE"
      command: ["sh", "-ec"]
      args: ["mkdir -p /recovery/data; chown -R $LAB_UID:$LAB_GID /recovery/data"]
      securityContext:
        runAsUser: 0
      volumeMounts:
        - name: target
          mountPath: /recovery
  containers:
    - name: toolbox
      image: "$LAB_IMAGE"
      command: ["sh", "-c", "sleep infinity"]
      volumeMounts:
        - name: target
          mountPath: /recovery
        - name: repo
          mountPath: /repo
          readOnly: true
        - name: config
          mountPath: /etc/pgbackrest/pgbackrest.conf
          subPath: pgbackrest.conf
          readOnly: true
  volumes:
    - name: target
      persistentVolumeClaim:
        claimName: day4-lab-restore-data
    - name: repo
      persistentVolumeClaim:
        claimName: backup-lab-repo
    - name: config
      configMap:
        name: backup-lab-config
YAML
kubectl apply -f day4-lab-restore-pod.yaml
kubectl -n postgres-ha wait --for=condition=Ready pod/day4-lab-restore --timeout=180s
```

If wait times out, inspect `kubectl -n postgres-ha describe pod day4-lab-restore` and both PVCs; do not run restore until Ready.

4. Confirm the recovery path is empty and repository readable

```
kubectl -n postgres-ha exec day4-lab-restore -c toolbox -- \
  sh -c 'test ! -e /recovery/data/PG_VERSION'
kubectl -n postgres-ha exec day4-lab-restore -c toolbox -- \
  gosu postgres pgbackrest --stanza=lab info
```

Lab 6 | Restore and verify PostgreSQL starts

5. Restore the most recent valid backup set

```
kubectl -n postgres-ha exec day4-lab-restore -c toolbox -- \
  gosu postgres pgbackrest --stanza=lab \
  --pg1-path=/recovery/data restore
kubectl -n postgres-ha exec day4-lab-restore -c toolbox -- \
  cat /recovery/data/PG_VERSION
```

Expected: restore command end: completed successfully; PG_VERSION shows 18. Restoring the latest set can require earlier full/differential/incremental sets and WAL. PG_VERSION alone does not prove recovery.

6. Start the isolated restored database and run SQL

```
kubectl -n postgres-ha exec day4-lab-restore -c toolbox -- \
  gosu postgres pg_ctl -D /recovery/data \
  -o "-c listen_addresses='' -c port=5544 -c unix_socket_directories=/tmp" \
  -l /recovery/data/day4-recovery.log -w start
kubectl -n postgres-ha exec day4-lab-restore -c toolbox -- \
  gosu postgres psql -h /tmp -p 5544 -U postgres \
  -d postgres -c 'SELECT current_database(), pg_is_in_recovery();'
kubectl -n postgres-ha exec day4-lab-restore -c toolbox -- \
  gosu postgres psql -h /tmp -p 5544 -U postgres \
  -d postgres -c 'SELECT datname FROM pg_database ORDER BY datname;'
```

Expected: SQL connects; pg_is_in_recovery() should become false after normal recovery. This server listens only on a Unix socket inside the disposable pod. If startup fails, inspect the log below and stop; do not point the operator at the recovery PVC.

```
kubectl -n postgres-ha exec day4-lab-restore -c toolbox -- \
  sh -c 'tail -80 /recovery/data/day4-recovery.log'
```

7. Stop the isolated server when finished

```
kubectl -n postgres-ha exec day4-lab-restore -c toolbox -- \
  gosu postgres pg_ctl -D /recovery/data -m fast -w stop
```

Keep day4-lab-restore-data while reviewing your results. Deleting a local-path PVC can delete its recovery data. The source backup-lab-0 and Patroni database pods are not restarted by this exercise.

Lab 7 | Point in time recovery in the lab database

PITR needs a full backup taken before the test change and archived WAL up to the chosen UTC target. It uses its own NEW PVC, day4-lab-pitr-data. The earlier day4-lab-restore-data is never reused. The following operations change only the disposable standalone backup-lab-0 database.

1. Take a new full backup, insert good data, capture time

```
kubectl -n postgres-ha exec backup-lab-0 -- \
  gosu postgres pgbackrest --stanza=lab --type=full backup
kubectl -n postgres-ha exec backup-lab-0 -- \
  gosu postgres psql -U postgres -d postgres \
  -c 'CREATE TABLE IF NOT EXISTS dr_day4_demo
    (id integer PRIMARY KEY, note text);'
kubectl -n postgres-ha exec backup-lab-0 -- \
  gosu postgres psql -U postgres -d postgres \
  -c "INSERT INTO dr_day4_demo VALUES (280926, 'keep this')
    ON CONFLICT (id) DO UPDATE SET note = EXCLUDED.note;"
TARGET_TIME=$(kubectl -n postgres-ha exec backup-lab-0 -- \
  gosu postgres psql -U postgres -d postgres -Atqc \
  "SELECT to_char(clock_timestamp() AT TIME ZONE 'UTC',
    'YYYY-MM-DD HH24:MI:SS.US') || '+00';")
echo "$TARGET_TIME"
test -n "$TARGET_TIME"
```

The UTC target is after the good insert and before the delete. Keep TARGET_TIME in this same Bash terminal. If you open a new terminal, copy the recorded time into TARGET_TIME manually before restore.

2. Delete only the test row and switch WAL

```
kubectl -n postgres-ha exec backup-lab-0 -- \
  gosu postgres psql -U postgres -d postgres \
  -c 'DELETE FROM dr_day4_demo WHERE id = 280926;'
kubectl -n postgres-ha exec backup-lab-0 -- \
  gosu postgres psql -U postgres -d postgres \
  -c 'SELECT pg_switch_wal();'
kubectl -n postgres-ha exec backup-lab-0 -- \
  gosu postgres pgbackrest --stanza=lab check
kubectl -n postgres-ha exec backup-lab-0 -- \
  gosu postgres pgbackrest --stanza=lab info
```

The current standalone database no longer has the row. PITR must bring it back by stopping recovery before the delete. If check fails or WAL needed for the target is missing, stop; no complete PITR is possible.

3. Create a fresh target and its separate toolbox

```
sed 's/day4-lab-restore-data/day4-lab-pitr-data/g' \
  day4-lab-restore-pvc.yaml > day4-lab-pitr-pvc.yaml
sed -e 's/day4-lab-restore-data/day4-lab-pitr-data/g' \
  -e 's/day4-lab-restore/day4-lab-pitr/g' \
  day4-lab-restore-pod.yaml > day4-lab-pitr-pod.yaml
kubectl apply -f day4-lab-pitr-pvc.yaml
kubectl apply -f day4-lab-pitr-pod.yaml
kubectl -n postgres-ha wait --for=condition=Ready \
  pod/day4-lab-pitr --timeout=180s
kubectl -n postgres-ha exec day4-lab-pitr -c toolbox -- \
  sh -c 'test ! -e /recovery/data/PG_VERSION'
```

Gate: check the generated filenames and manifest claimName before apply. Never alter the active data-backup-lab-0 claim or any pgdata-acid-postgres-ha claim.

Lab 7 | PITR restore and SQL proof

4. Restore to the captured UTC target on the new empty PVC

```
test -n "$TARGET_TIME"
kubectrl -n postgres-ha exec day4-lab-pitr -c toolbox -- \
  gosu postgres pgbackrest --stanza=lab \
    --pg1-path=/recovery/data --type=time \
    --target="$TARGET_TIME" --target-action=promote restore
kubectrl -n postgres-ha exec day4-lab-pitr -c toolbox -- \
  cat /recovery/data/PG_VERSION
```

Expected: restore completes and PG_VERSION shows 18. The restore command writes recovery settings into this isolated data directory; SQL proof requires starting that isolated instance. An incomplete WAL archive may prevent recovery; inspect the startup log if needed.

```
kubectrl -n postgres-ha exec day4-lab-pitr -c toolbox -- \
  gosu postgres pg_ctl -D /recovery/data \
    -o "-c listen_addresses='' -c port=5545 -c unix_socket_directories=/tmp" \
    -l /recovery/data/day4-pitr.log -w start
kubectrl -n postgres-ha exec day4-lab-pitr -c toolbox -- \
  gosu postgres psql -h /tmp -p 5545 -U postgres \
    -d postgres \
    -c 'SELECT id, note FROM dr_day4_demo WHERE id = 280926;'
kubectrl -n postgres-ha exec day4-lab-pitr -c toolbox -- \
  gosu postgres pg_ctl -D /recovery/data -m fast -w stop
```

Expected: the recovered row says keep this. This verifies that the later delete was not replayed. Keep the recovery PVC for review; a fresh PITR attempt requires another new empty PVC, never a running PostgreSQL directory.

Lab 8 | Live Patroni backup status and next design

1. Inspect the actual live cluster

```
kubectl -n postgres-ha get pods -o wide
kubectl -n postgres-ha get pvc
kubectl -n postgres-ha get postgresql acid-postgres-ha \
-o jsonpath='{.spec.dockerImage}{"\n"}'
```

Expected: all three pods Ready. The configured image in your dry run is `docker.io/eswaribala/ericpostgrerepo2026:pg18`. Zalando generates a StatefulSet with update strategy `OnDelete` here; `kubectl rollout status` does not validate that strategy. Inspect actual pod image and readiness instead.

```
for pod in acid-postgres-ha-0 acid-postgres-ha-1 acid-postgres-ha-2; do
  echo "$pod"
  kubectl -n postgres-ha get pod "$pod" \
  -o jsonpath='{.spec.containers[0].image}{"\n"}'
  kubectl -n postgres-ha get pod "$pod" \
  -o jsonpath='{.status.containerStatuses[0].ready}{"\n"}'
done
```

2. Understand what is still required for live physical backups

Before claiming protection of `acid-postgres-ha`, provision a separate durable repository accessible to the Patroni pods, configure `pgBackRest` on the actual primary and replicas, enable and verify WAL archiving, take a full backup, run `pgbackrest check/info` for that stanza, and restore to an isolated environment. The local-path backup-lab-repo is owned by the standalone lab and is not a failover-safe shared repository for the live three-pod cluster.

Your non-GKE Kubernetes cluster needs an approved cloud identity mechanism if you select GCS. A bucket existing in GCP or a Google login on the Ubuntu host does not authenticate Kubernetes pods. With service-account key creation blocked, use approved Workload Identity Federation or another repository architecture after confirming support for the actual `pgBackRest` image and operator configuration. No unverified `gcloud` key or custom image rebuild instructions are included here.

Lab 9 | GCS preparation and the blocked access step

This optional section preserves the GCS lab setup from the original manual. Its bucket commands run on Ubuntu using your selected gcloud account and project. They do not grant your Kubernetes pods cloud access. Do not configure a GCS pgBackRest stanza or switch the live WAL archive until identity is solved and tested.

```
gcloud auth list
gcloud config get-value project
gcloud projects list
gcloud billing projects describe \
  "$(gcloud config get-value project)"
```

The project ID is the command output, not its display name. Set the correct project with gcloud config set project PROJECT_ID if needed. A missing or disabled billing account can prevent bucket creation.

```
PROJECT_ID=$(gcloud config get-value project)
GCS_BUCKET=postgres-ha-backups-parameswari-2026
echo "$PROJECT_ID"
echo "$GCS_BUCKET"
gcloud storage buckets describe "gs://$GCS_BUCKET"
```

If describe succeeds, your bucket exists; skip create. If it says not found and you have billing/permissions, the next command attempts to create the example name. Bucket names are global: choose a unique name if that name belongs to another project.

```
gcloud storage buckets create "gs://$GCS_BUCKET" \
  --project="$PROJECT_ID" --location=asia-south1 \
  --default-storage-class=STANDARD \
  --uniform-bucket-level-access \
  --public-access-prevention
gcloud storage buckets describe "gs://$GCS_BUCKET"
```

If the earlier describe succeeded, do not run the create command. Your organization rejected JSON key creation with FAILED_PRECONDITION due to constraints/iam.disableServiceAccountKeyCreation. gcloud auth login on Ubuntu does not provide credentials to pods. Set up an approved external Kubernetes identity federation or another failover-safe repository, confirm the selected pgBackRest version can use it, and perform a pod-level identity check before taking live physical backups.

Meaning of shell commands

NAME=value sets a variable for this terminal. echo "\$NAME" prints its current value. unset NAME clears that variable, not your GCP project or bucket. gcloud config set project PROJECT_ID changes the CLI-selected project; gcloud auth login changes the signed-in CLI account. Neither operation edits the Kubernetes PostgreSQL custom resource.

Reference, troubleshooting and completion checks

What each backup covers

lab full/diff/incr: backup-lab-0 database and WAL in its local PVC repository. banking.dump: one live Patroni database at dump time on your Ubuntu computer. Neither is a verified live-cluster physical backup with off-node disaster recovery.

Common failures

ERROR [031] pgBackRest must not run as root: use gosu postgres. kubectl says executable '--' not found: remove the extra -- after postgres. kubectl edit cancelled: no saved CR change; check spec.dockerImage and edit again. kubectl rollout status unsupported: check OnDelete pod readiness and actual images instead. GCS FAILED_PRECONDITION: service account key creation blocked by organization policy; stop that path. PVC Pending: WaitForFirstConsumer may wait for scheduling; inspect pod events. PG_VERSION exists in a recovery target: do not rerun restore on it; use a new isolated PVC after completing the current drill.

Lab success checklist

Lab 0-4: identify a live primary, take a logical dump, run optional base backup, record WAL archiving, and verify all three pod images. Lab 5-7: pgbackrest info status ok; full/diff/incr visible; check successful; isolated recovery PVC separate from all existing database PVCs; restored PostgreSQL 18 starts, SQL succeeds, and the PITR test row is present. Lab 8-9: understand that live Patroni physical backup still requires a supported durable repository and verified pod identity.

Sources and scope

Official pgBackRest user guide: <https://pgbackrest.org/user-guide.html>

Google Cloud IAM organization policy and Workload Identity Federation: <https://cloud.google.com/iam/docs/service-account-creds> and <https://cloud.google.com/iam/docs/workload-identity-federation>

Commands in this manual were written for the environment shown in your screenshots. Execution against your cluster and the isolated restore outcome must still be verified there.